

Mars Horizon × Archipelago — Prompt & Roadmap

Oct 4, 2026 · @Haiko

Contexte

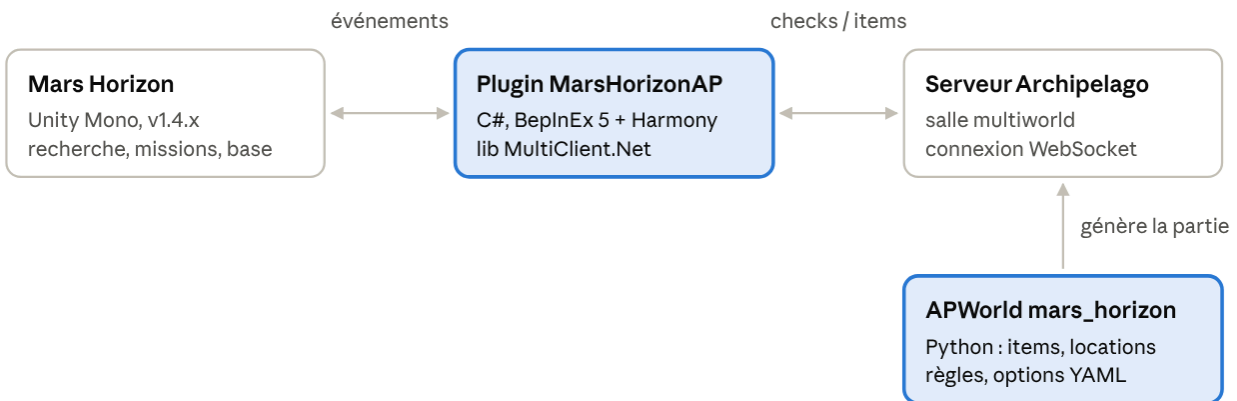
Le projet est faisable : Mars Horizon est un jeu Unity (build Mono, Windows) déjà moddé avec BepInEx 5 + Harmony, et aucun APWorld n'existe pour lui à ce jour.

- **Le jeu** : simulation de gestion d'agence spatiale (Auroch Digital, 2020). On construit une base, on recherche un arbre technologique, on assemble des fusées et on enchaîne des missions au tour par tour jusqu'au premier humain sur Mars, en course contre des agences rivales. [Auroch Digital](#)
- **Modding prouvé** : les mods de Sheep-y (Informed, Simple Mod Loader...) tournent sous BepInEx 5.4 et visent la version 1.4.1 du jeu, build Mono uniquement. Le code du jeu vit dans `Assembly-CSharp.dll`, namespace principal `Astronautica`. [Nexus Mods](#) · [GitHub Sheep-y](#)
- **Archipelago** : un monde AP = un APWorld Python (items, locations, régions, règles, options) + un client dans le jeu qui envoie les « checks » et reçoit les items. Le modèle le plus proche est l'[Outer Wilds Randomizer](#) (Unity, C#, très bien documenté).
- **Point d'attention** : le jeu est une progression assez linéaire (ères successives). Le randomizer doit donc découper la progression en verrous (technos, bâtiments, destinations) pour créer de vraies branches.

Architecture technique

Deux briques à écrire : un APWorld Python (génération) et un plugin BepInEx C# (dans le jeu) qui parle au serveur via la bibliothèque officielle Archipelago.MultiClient.Net.

Deux livrables à coder : le plugin C# et l'APWorld Python



architecture · 4 composants, 2 à coder

- **APWorld** (`worlds/mars_horizon/`) : `__init__.py` (classe `World`), `items.py`, `locations.py`, `regions.py`, `rules.py`, `options.py`, `docs/`. Testé avec le framework de tests d'Archipelago.
- **Plugin C#** : patches Harmony qui (1) bloquent les technos / bâtiments / destinations non reçus, (2) détectent les fins de recherche, constructions et missions pour envoyer les checks, (3) appliquent les items reçus, y compris hors ligne via une file d'attente sauvegardée.
- **Persistance** : l'index du dernier item reçu est stocké dans la sauvegarde (ou un fichier à côté), pour ne rien donner deux fois après un rechargement.
- **UI minimale** : fenêtre de connexion (hôte, slot, mot de passe), notifications d'items, mini-tracker des checks restants.

Concept du randomizer

Principe : chaque techno, bâtiment et destination devient un item Archipelago ; chaque premier succès de mission, recherche terminée ou jalon de la course spatiale devient une location. Les listes exactes seront extraites du code du jeu en phase 1.

Catégorie	Items (ce qu'on reçoit)	Locations (ce qui envoie un check)
Recherche	Chaque nœud de l'arbre techno (fusées, modules, payloads, capsules)	Terminer la recherche d'un nœud (le nœud lui-même n'est plus débloquent)
Base	Permis de construction par bâtiment ou niveau d'amélioration	Construire / améliorer un bâtiment pour la première fois

Catégorie	Items (ce qu'on reçoit)	Locations (ce qui envoie un check)
Destinations	Accès aux cibles : orbite terrestre, Lune, Vénus, Mars...	Premier succès d'une mission vers chaque cible
Jalons	—	« Premières » de la course spatiale (premier satellite, premier humain en orbite, alunissage...)
Ressources (filler)	Paquets d'argent, de science, de soutien public	—
Pièges (option)	Panne de pad, perte de soutien, retard de construction	—

Objectifs (option YAML) : premier humain sur Mars (défaut) · N jalons atteints · base martienne établie.

Options envisagées : mélanger ou non les bâtiments, inclure les pièges, pourcentage de filler, agence de départ, techno de départ garantie (sounding rocket + pad de lancement), mode « DeathLink » = un échec de lancement critique envoie un DeathLink.

Règle de logique de base : une mission vers X exige la destination X + une fusée capable (booster + étages requis) + le payload adapté + le bâtiment de lancement. Ces règles vivent dans `rules.py` de l'APWorld.

Le prompt (à coller dans Claude Code)

À lancer à la racine d'un dépôt vide, avec le dossier du jeu accessible. Une phase à la fois : on valide chaque livrable avant de passer à la suivante.

```
# Rôle
Tu es un développeur expert en modding Unity (BepInEx 5, HarmonyX, C#/.NET Framework 4.x)
et en développement de mondes Archipelago (APWorld Python, Archipelago.MultiClient.Net).
Tu travailles avec moi, étape par étape, sur un randomizer Archipelago pour Mars Horizon.

# Contexte
- Jeu : Mars Horizon (Auroch Digital, 2020), Unity build Mono, Windows, version 1.4.x.
- Code du jeu : <CHEMIN_DU_JEU>/Mars Horizon_Data/Managed/Assembly-CSharp.dll (namespace principal : Astronautica).
- Référence de modding existante : https://github.com/Sheep-y/MarsHorizonMods
- Référence de randomizer Unity :
```

<https://github.com/Ixrec/OuterWildsArchipelagoRandomizer>

- Doc Archipelago :

<https://github.com/ArchipelagoMW/Archipelago/tree/main/docs>
(world api.md, adding games.md, network protocol.md).

Objectif

Livrer deux composants :

1. Un APWorld `mars\_horizon` (Python) : items, locations, régions, règles de logique, options YAML, tests unitaires, documentation joueur.
2. Un plugin BepInEx `MarsHorizonAP` (C#) : connexion au serveur, verrouillage des technos/bâtiments/destinations non reçus, envoi des checks, réception et application des items, persistance dans la sauvegarde, petite UI (connexion, notifications, tracker).

Design du randomizer

- Items : nœuds de l'arbre technologique, permis de bâtiments, accès aux destinations, filler (argent, science, soutien public), pièges optionnels.
- Locations : fin de chaque recherche, première construction/amélioration de chaque bâtiment, premier succès de mission par destination, jalons de la course spatiale.
- Objectif par défaut : premier humain sur Mars (configurable).
- Logique : une mission vers X exige l'accès à X + une fusée capable + le payload adapté + l'infrastructure de lancement. Toujours garantir un début jouable (sphère 1 non vide).

Règles de travail

- Commence TOUJOURS par explorer avant de coder : décompile Assembly-CSharp.dll (ilspycmd) et documente dans `docs/game-internals.md` les classes, méthodes et données clés (recherche, construction, missions, sauvegarde, jalons).
- N'invente jamais un nom de classe ou de méthode : cite le fichier décompilé d'origine.
- Les listes d'items/locations sont générées à partir des données du jeu (script d'extraction en `tools/`), pas tapées à la main.
- Garde une table d'IDs stable (base d'ID unique, jamais réutilisée).
- Petits commits, un par étape. Explique ce que tu vas faire avant chaque phase et attends ma validation entre les phases.

- Si une info manque ou qu'un choix de design est ambigu, pose-moi la question.

Première tâche

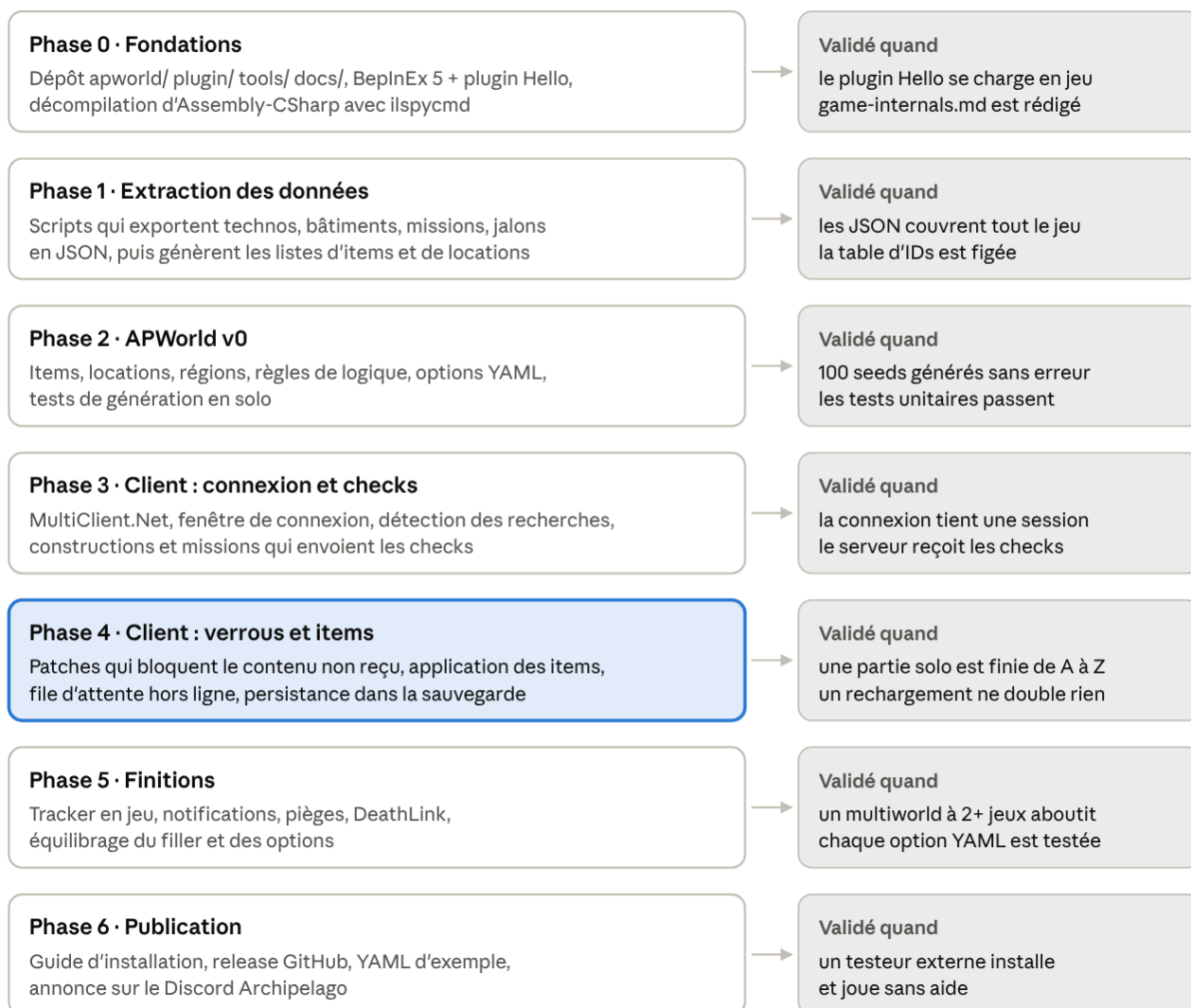
Phase 0 de la roadmap : mets en place le dépôt (structure apworld/ + plugin/ + tools/ + docs/),
vérifie que BepInEx charge un plugin "Hello" dans le jeu, puis lance l'exploration du code
et rends-moi `docs/game-internals.md`.

Remplace `<CHEMIN_DU_JEU>` par ton dossier Steam (souvent `C:\Program Files (x86)\Steam\steamapps\common\Mars Horizon`).

Roadmap

Sept phases, chacune fermée par un critère vérifiable ; on ne passe à la suivante qu'une fois le critère atteint. Le jalon décisif est la phase 4 : une partie solo randomisée jouable du début à la fin.

7 phases : la partie solo jouable de bout en bout arrive en phase 4



roadmap · 7 phases et leurs critères de validation

Conseil de rythme : lancer Claude Code sur une phase par session, avec le prompt ci-dessus et le rappel « nous sommes en phase N ».

- ☐ Phase 0 · Fondations
- ☐ Phase 1 · Extraction des données
- ☐ Phase 2 · APWorld v0
- ☐ Phase 3 · Client : connexion et checks
- ☐ Phase 4 · Client : verrous et items
- ☐ Phase 5 · Finitions
- ☐ Phase 6 · Publication

Risques et points ouverts

Risque	Impact	Parade
Progression trop linéaire (ères successives)	Peu de vraies branches, joueur bloqué souvent	Mélanger aussi bâtiments et destinations ; garantir 3-5 checks en sphère 1
Mise à jour du jeu qui casse les patches	Plugin inutilisable	Viser une version précise, vérifier la version au lancement
Agences rivales qui « volent » les jalons	Location manquée définitivement	Faire compter le jalon même si un rival l'atteint avant (ou option pour les exclure)
Événements aléatoires du jeu (missions, budgets)	Logique non garantie	Ne mettre en logique que ce qui est toujours accessible
Sauvegarde et rechargement	Items en double ou perdus	Index de réception persistant + tests dédiés en phase 4

Questions à trancher avant la phase 2

- ☐ Une seule agence jouable ou toutes (chaque agence a sa base et ses véhicules) ?
- ☐ Le DLC éventuel est-il inclus ?
- ☐ Granularité des items : chaque nœud techno, ou des « paquets » par branche ?
- ☐ Les jalons des rivaux comptent-ils comme locations ?

Sources

- [Mars Horizon — Auroch Digital](#)
- [Simple Mod Loader / guide BepInEx — Nexus Mods](#)
- [Sheep-y/MarsHorizonMods — GitHub](#)
- [BepInEx — GitHub](#)
- [Archipelago — GitHub](#)
- [Outer Wilds Archipelago Randomizer — GitHub](#)